



Earliest Deadline First



Luís Miguel Pinho

On-line scheduling with dynamic priorities

- ▶ Order of execution of tasks is determined during execution
 - ▶ Based on properties calculated on-line, which specify a “dynamic priority”
 - ▶ The ready queue is updated every time properties change
- ▶ Advantages
 - ▶ Easy to scale to many tasks
 - ▶ Easy to accommodate tasks with dissimilar periods (and sporadics)
- ▶ Disadvantages
 - ▶ Higher runtime complexity and overhead
 - ▶ Unknown behavior under overloads
 - ▶ not possible to know beforehand which tasks are delayed

On-line scheduling with dynamic priorities

▶ Examples

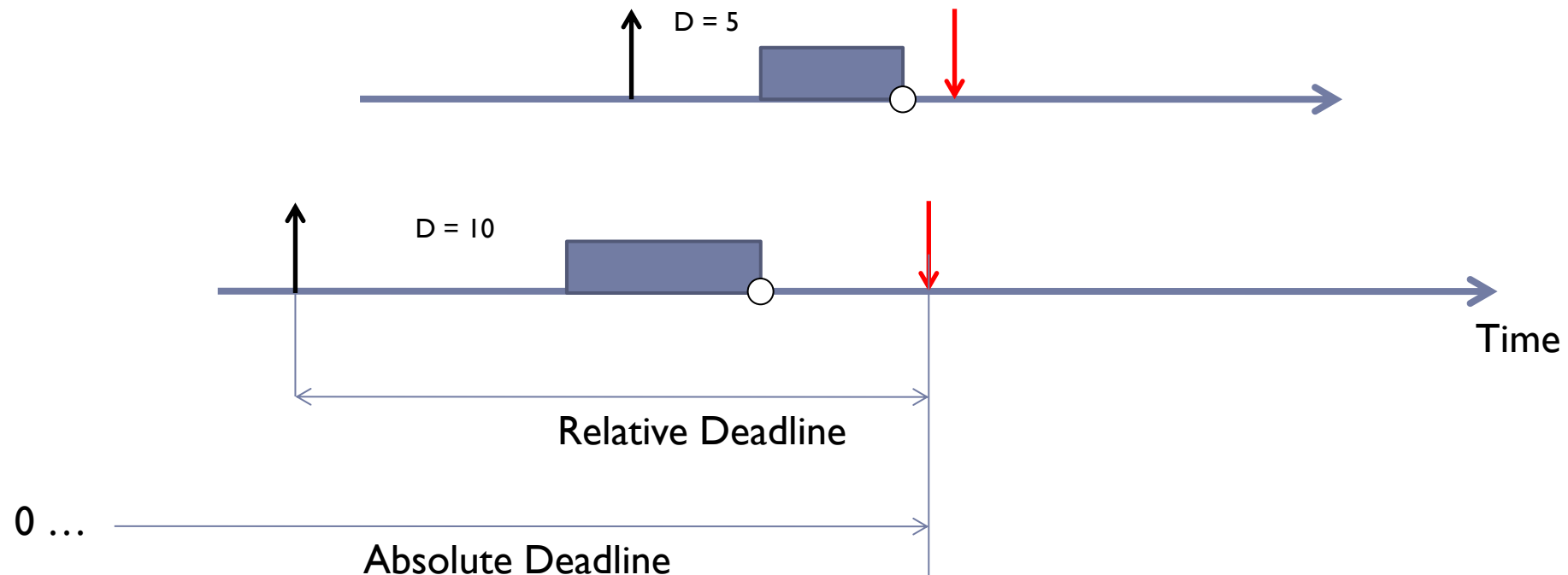
- ▶ Earliest Deadline First (EDF)
 - ▶ Next to execute is the task with the nearer absolute deadline
 - ▶ This is the most used, which we will consider in the class
- ▶ Least Laxity First (LLF) / Least Slack First (LSF)
 - ▶ Next to execute is the task with the smaller spare time
- ▶ First Come First Serve (FCFS)
 - ▶ Next to execute is the task with a larger waiting time

Earliest Deadline First (EDF)

- ▶ **Fully on-line algorithm**
 - ▶ The runnable tasks are executed in the order determined by their absolute deadlines
 - ▶ The next task to run is always the one with the shortest (nearest) deadline
 - ▶ Although it is usual to know the relative deadlines of each process (e.g. 25ms after release), the absolute deadlines are computed at run time and hence the scheme is described as dynamic
- ▶ **It allows better utilization in the system, although in overloads there is an unpredictably issue**
 - ▶ No way to know which task gets a deadline miss – in FPS is always the lowest priority one

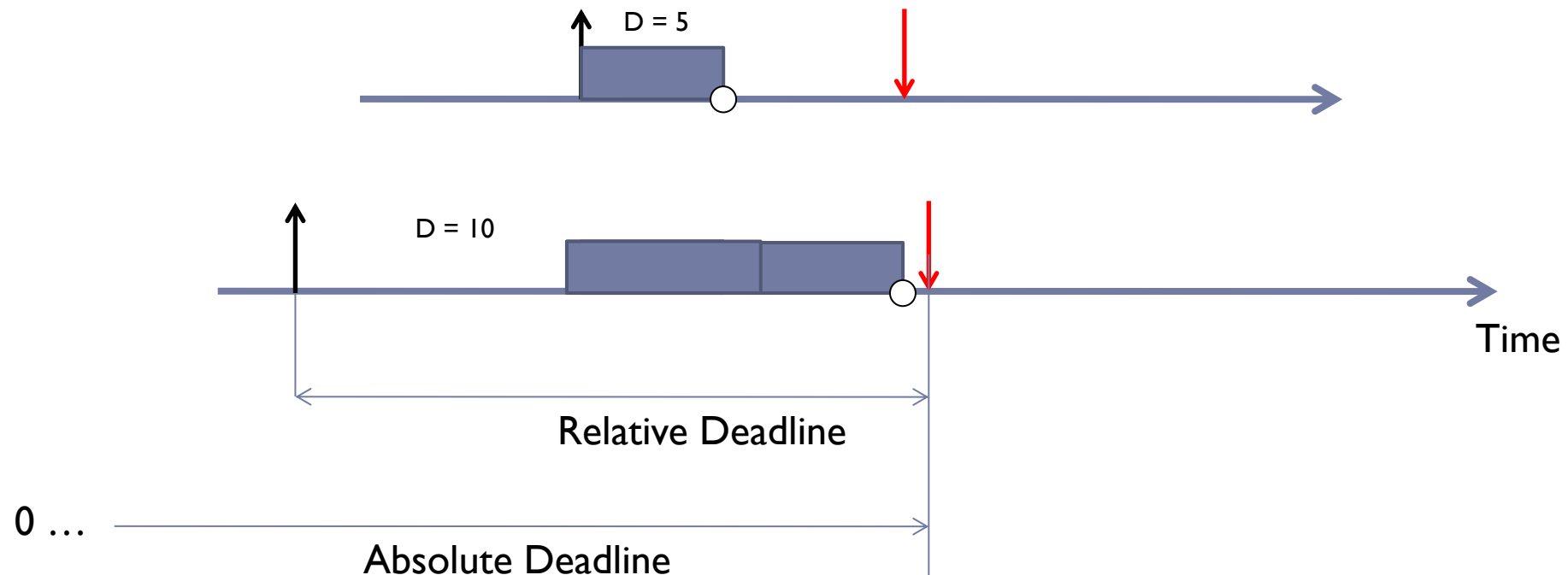
Earliest Deadline First (EDF)

- ▶ Instant of arrival is fundamental



Earliest Deadline First (EDF)

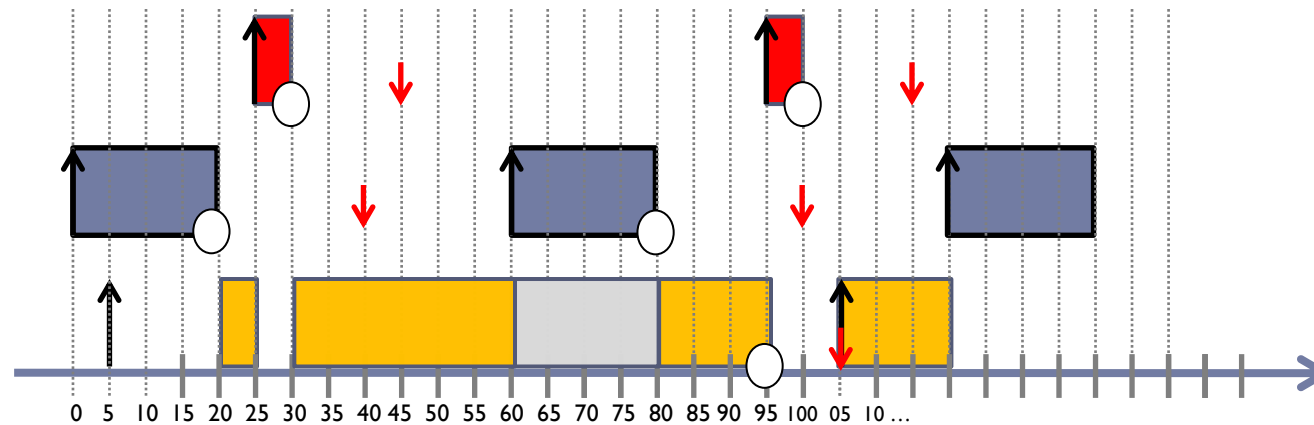
- ▶ Instant of arrival is fundamental



Earliest Deadline First (EDF)

► EDF Example

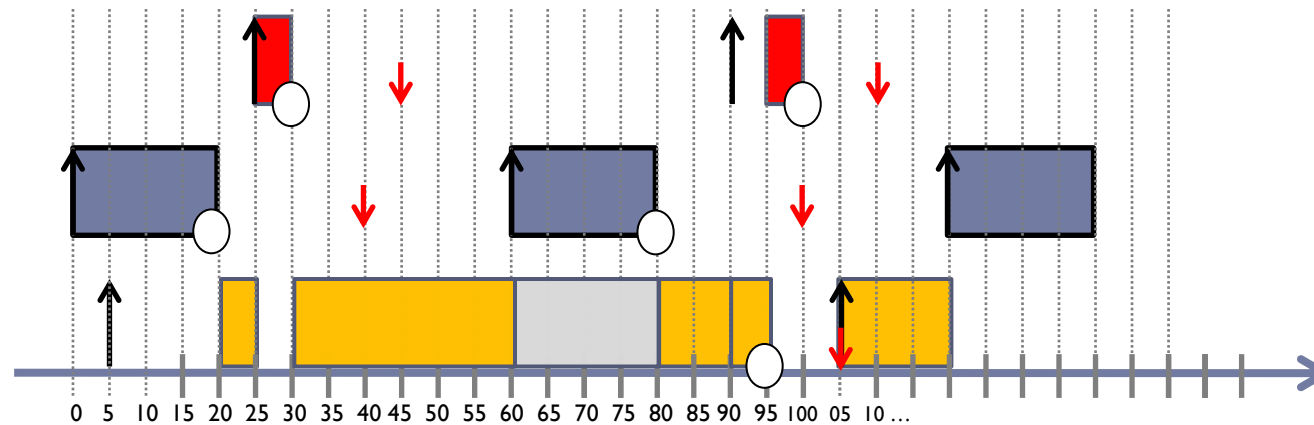
	C (ms)	T (ms)	D (ms)	Priority	Arrival instant
 Control	20	60	40	-	0
 Alarm	5	70	20	-	25
 Logger	50	100	100	-	5



Earliest Deadline First (EDF)

► EDF Example

	C (ms)	T (ms)	D (ms)	Priority	Arrival instant	
	Control	20	60	40	-	0
	Alarm	5	65	20	-	25
	Logger	50	100	100	-	5



Earliest Deadline First (EDF)

► Utilization based tests

► For pre-emptive FPS ($D=T$)

- If $U > 1 \Rightarrow$ The system is never schedulable, since CPU utilization is higher than 100%
- If $U \leq n(2^{1/n} - 1) \Rightarrow$ The system is schedulable
- Between these values no conclusion can be made

► For pre-emptive EDF ($D=T$)

- If $U > 1 \Rightarrow$ The system is never schedulable, since CPU utilization is higher than 100%
- If $U \leq 1 \Rightarrow$ The system is always schedulable

► For pre-emptive EDF ($D < T$)

- If $U^* > 1 \Rightarrow$ The system is never schedulable (U^* is density $\Rightarrow C_i/D_i$)
- If $U^* \leq 1 \Rightarrow$ The system is always schedulable

Earliest Deadline First (EDF)

► Utilization based tests

- Remember pre-emptive FPS ($D=T$)

	C (ms)	T=D(ms)	U(Ci/Ti)
τ_1	2	10	0,20
τ_2	9	15	0,60
τ_3	1	25	0,04
	$U (\sum C_i / T_i) =$		84%

$$3(2^{1/3} - 1) \approx 78\%$$

$$U = \sum_{i=1}^n \frac{C_i}{T_i} = 84\% > 78\%$$

- With EDF this taskset is guaranteed to be schedulable

Earliest Deadline First (EDF)

- ▶ **Non-preemptive case**
 - ▶ With dynamic priorities, the non-preemptive case becomes more complex, as the concept of blocking is also dynamic
 - ▶ A blocking task is any task that may have a nearer absolute deadline
 - ▶ It is necessary to consider the processor load within a synchronous busy period
 - ▶ The longest period of continuous CPU load
 - ▶ But it is then necessary to check all cases within such period
- ▶ **What about response time analysis?**
 - ▶ EDF response time analysis is more complex than FPS since it is not known what job suffers the maximal interference (there is no concept such as the critical instant)
 - ▶ It is known that the worst-case response time occurs within the synchronous busy period
 - ▶ Same issue as for non-preemptive case
- ▶ **This is out of scope of the course**

Earliest Deadline First (EDF)

- ▶ EDF is considered to be fair
 - ▶ In the sense that all tasks see their priority raised as they are delayed close to their deadline, independently of their period or other static parameter
 - ▶ There are no “more important” tasks
 - ▶ This also means that in an overload it is not possible to know which tasks will fail deadlines
- ▶ EDF also avoids preemptions when tasks come closer to their deadlines
 - ▶ Means that faster appearing tasks (smaller periods) are heavily influenced by current executing situation leading to higher jitter

Earliest Deadline First (EDF)

► Exercise

- Trace the execution of the following taskset, with FPS and EDF, for 100 ms

	C (ms)	T=D (ms)	Arrival
τ_1	10	30	0
τ_2	10	40	0
τ_3	21	60	0

Earliest Deadline First (EDF)

- ▶ Let's check with SimSo

- ▶ <https://projects.laas.fr/simso/simso-web/#/configuration>

- ▶ Use advanced mode and task additional fields in FPS to configure priority

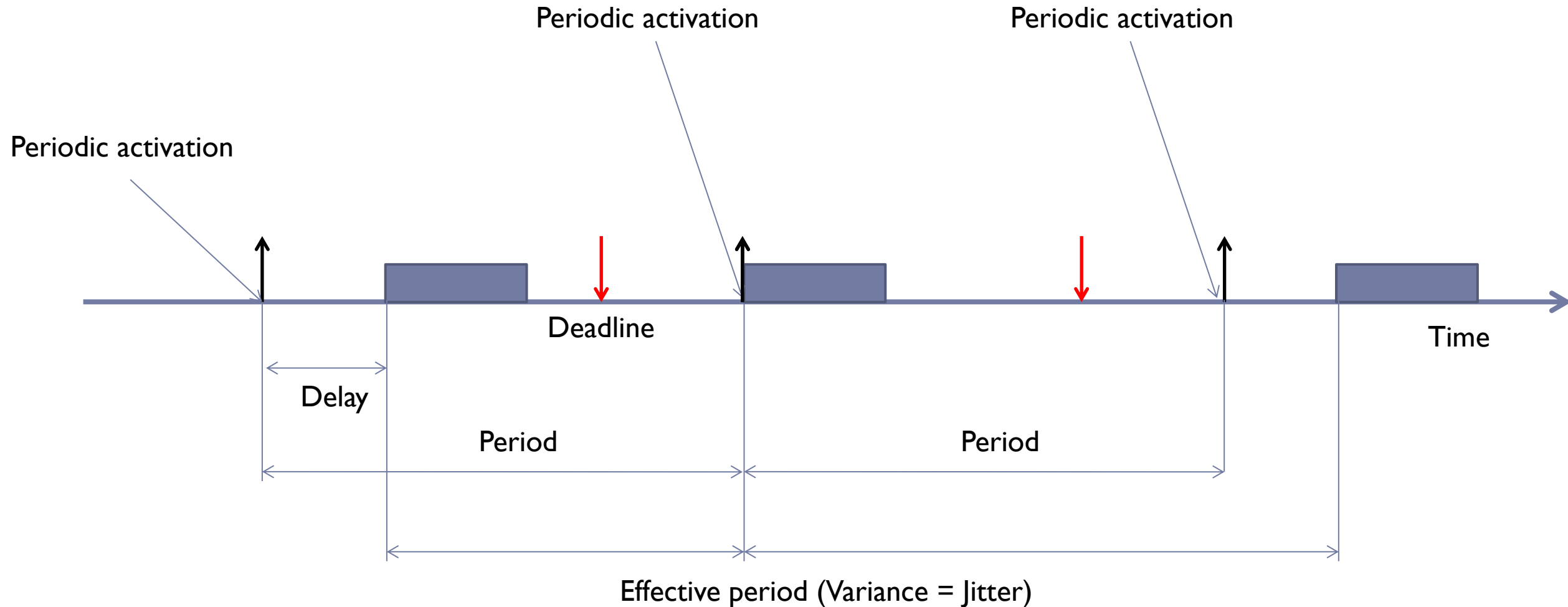
- ▶ JSON

- ▶ configuration_trace_exercise_2_FPS.json

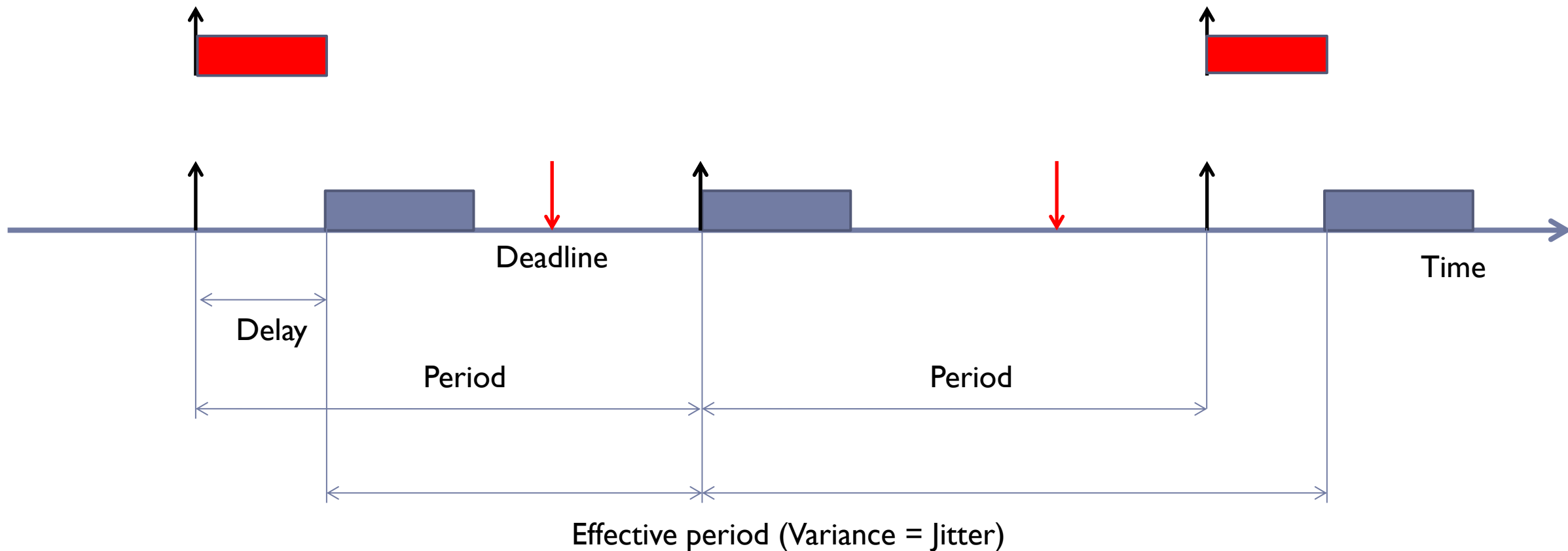
- ▶ configuration_trace_exercise_2_EDF.json

- ▶ configuration_trace_exercise_2_FPS_offset.json

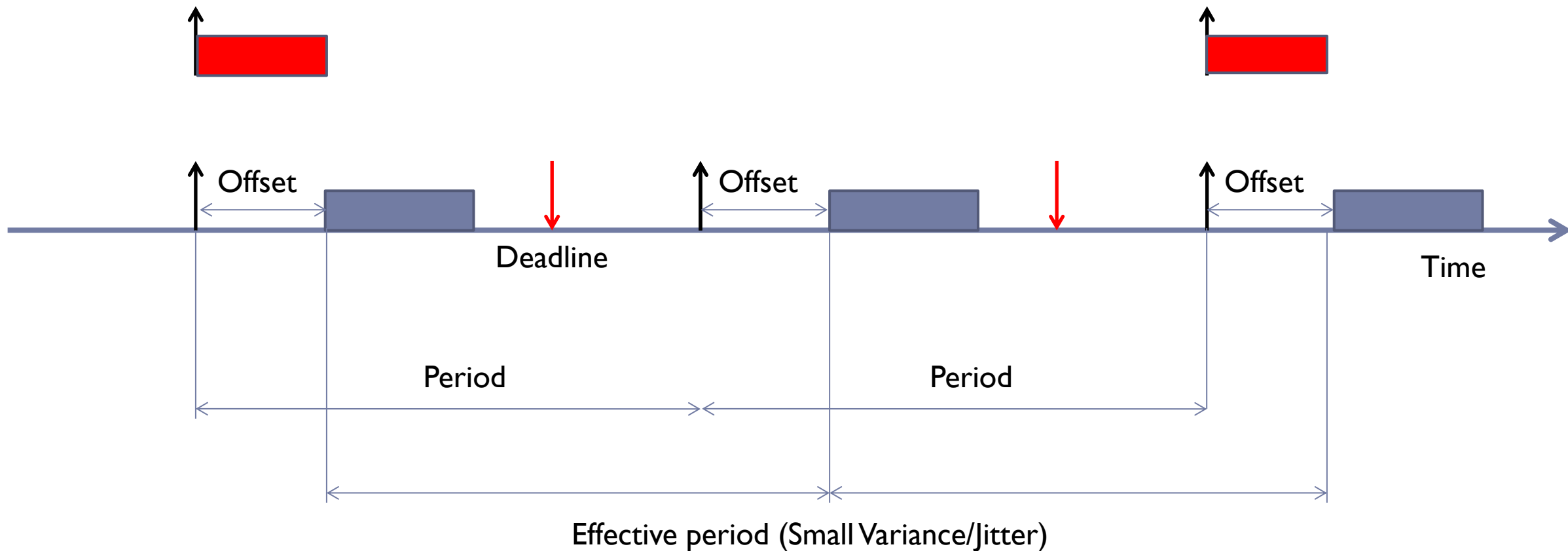
Improving Periodic Behaviour with Offsets



Improving Periodic Behaviour with Offsets



Improving Periodic Behaviour with Offsets



Dynamic Scheduling

► Exercise

- Check if the following taskset is schedulable under FCFS (First Come First Serve)

	C (ms)	T=D (ms)
τ_1	25	50
τ_2	10	60
τ_3	20	80

Dynamic Scheduling

► Exercise

- EDF again, preemptive, what is the minimum period for task τ_1 , so that the following taskset is schedulable

	C (ms)	T=D (ms)
τ_1	25	?
τ_2	10	60
τ_3	20	80

Hierarchical Scheduling

- ▶ There are systems which support a mix of scheduling approaches
- ▶ The most widely used is hierarchical scheduling, with more than one scheduling decision being taken
 - ▶ First, priority levels are assigned, and the ready task with the highest priority is selected
 - ▶ If there are more than one task in the same priority level, a second scheduling decision is taken, e.g. Earliest Deadline First, or Round-Robin with the priority
 - ▶ Several Operating Systems and languages support this model
 - ▶ Linux, Zephyr, Ada
 - ▶ More on this later